

Skill-04

Code:

```
#include <stdio.h>
#include <stdlib.h>
#include <string.h>
#include <ctype.h>

#define MAX_TOKENS 100
#define MAX_LEN 1000

typedef enum {
    WORD,
    PIPE,
    INPUT,
    OUTPUT,
    APPEND
} TokenType;

typedef struct {
    char value[100];
    TokenType type;
} Token;

typedef struct Node {
    char value[100];
    struct Node *left;
    struct Node *right;
} Node;

Token tokens[MAX_TOKENS];
int tokenCount = 0;

const char *typeName(TokenType type) {
    if (type == WORD) return "WORD";
    if (type == PIPE) return "PIPE";
    if (type == INPUT) return "INPUT";
    if (type == OUTPUT) return "OUTPUT";
    return "APPEND";
}

void addToken(const char *value, TokenType type) {
    if (tokenCount >= MAX_TOKENS)
        return;

    strcpy(tokens[tokenCount].value, value);
    tokens[tokenCount].type = type;
    tokenCount++;
}

void tokenize(char *input) {
    int i = 0;
```

```

while (input[i] != '\0') {
    if (isspace(input[i])) {
        i++;
        continue;
    }

    if (input[i] == '|') {
        addToken("|", PIPE);
        i++;
    }
    else if (input[i] == '<') {
        addToken("<", INPUT);
        i++;
    }
    else if (input[i] == '>') {
        if (input[i + 1] == '>') {
            addToken(">>", APPEND);
            i += 2;
        } else {
            addToken(">", OUTPUT);
            i++;
        }
    }
    else {
        char word[100];
        int j = 0;

        while (input[i] != '\0' &&
            !isspace(input[i]) &&
            input[i] != '|' &&
            input[i] != '<' &&
            input[i] != '>') {
            word[j++] = input[i++];
        }

        word[j] = '\0';
        addToken(word, WORD);
    }
}

int validate() {
    if (tokenCount == 0) {
        printf("Empty command\n");
        return 0;
    }

    if (tokens[0].type == PIPE ||
        tokens[0].type == INPUT ||
        tokens[0].type == OUTPUT ||
        tokens[0].type == APPEND) {
        printf("Syntax error: command cannot start with operator\n");
        return 0;
    }

    if (tokens[tokenCount - 1].type == PIPE ||
        tokens[tokenCount - 1].type == INPUT ||

```

```

    tokens[tokenCount - 1].type == OUTPUT ||
    tokens[tokenCount - 1].type == APPEND) {
    printf("Syntax error: command cannot end with operator\n");
    return 0;
}

for (int i = 0; i < tokenCount - 1; i++) {
    if (tokens[i].type == PIPE &&
        tokens[i + 1].type == PIPE) {
        printf("Syntax error: consecutive pipes\n");
        return 0;
    }

    if ((tokens[i].type == INPUT ||
        tokens[i].type == OUTPUT ||
        tokens[i].type == APPEND) &&
        tokens[i + 1].type != WORD) {
        printf("Syntax error: file name expected\n");
        return 0;
    }
}

return 1;
}

Node *createNode(char *value) {
    Node *node = malloc(sizeof(Node));

    strcpy(node->value, value);
    node->left = NULL;
    node->right = NULL;

    return node;
}

Node *buildTree() {
    Node *root = NULL;
    Node *current = NULL;

    for (int i = 0; i < tokenCount; i++) {
        if (tokens[i].type == PIPE) {
            Node *pipe = createNode("|");
            pipe->left = root;
            root = pipe;
            current = NULL;
        }
        else {
            Node *node = createNode(tokens[i].value);

            if (root == NULL) {
                root = node;
                current = node;
            }
            else if (current != NULL) {
                current->right = node;
                current = node;
            }
        }
    }
}

```

```

    }

    return root;
}

void printTokens() {
    printf("\nTokens:\n");

    for (int i = 0; i < tokenCount; i++) {
        printf("%d. %-10s %s\n",
            i + 1,
            tokens[i].value,
            typeName(tokens[i].type));
    }
}

void printTree(Node *root, int level) {
    if (root == NULL)
        return;

    for (int i = 0; i < level; i++)
        printf(" ");

    printf("%s\n", root->value);

    printTree(root->left, level + 1);
    printTree(root->right, level + 1);
}

void freeTree(Node *root) {
    if (root == NULL)
        return;

    freeTree(root->left);
    freeTree(root->right);
    free(root);
}

int main() {
    char input[MAX_LEN];

    printf("Mini Shell Parser\n");
    printf("Enter command: ");

    if (fgets(input, sizeof(input), stdin) == NULL)
        return 0;

    input[strcspn(input, "\n")] = '\0';

    tokenize(input);

    if (!validate())
        return 0;

    printTokens();

    Node *root = buildTree();

```

```

printf("\nParse Tree:\n");
printTree(root, 0);

printf("\nSyntax validation successful\n");
printf("Execution structure generated successfully\n");

freeTree(root);

return 0;
}

```

Screen Shots:

```

sohan_jadala@DESKTOP-0I7PGP2:~/2520080060_OSSP/2520080060_Skill$ gcc Skill_4.c -o Skill_4
sohan_jadala@DESKTOP-0I7PGP2:~/2520080060_OSSP/2520080060_Skill$ ./Skill_4
Mini Shell Parser
Enter command: ls-l

Tokens:
1. ls-l      WORD

Parse Tree:
ls-l

Syntax validation successful
Execution structure generated successfully

sohan_jadala@DESKTOP-0I7PGP2:~/2520080060_OSSP/2520080060_Skill$ gcc Skill_4.c -o Skill_4
sohan_jadala@DESKTOP-0I7PGP2:~/2520080060_OSSP/2520080060_Skill$ ./Skill_4
Mini Shell Parser
Enter command: ls | grep txt

Tokens:
1. ls      WORD
2. |      PIPE
3. grep    WORD
4. txt     WORD

Parse Tree:
|
ls

Syntax validation successful
Execution structure generated successfully
sohan_jadala@DESKTOP-0I7PGP2:~/2520080060_OSSP/2520080060_Skill$ |

```